Comparative Analysis of Machine Learning Models for Mushroom Classification

by

Dorra Ben Abdallah

January 2025

Tunis, TUNISIA.

2024–2025

List of Figures

Introduction

Mushroom classification is an essential problem in the field of food safety and agriculture. Identifying whether a mushroom is edible or poisonous based on its physical characteristics is crucial for preventing accidental poisoning. Some mushrooms, despite appearing harmless, contain toxic compounds that can be lethal if ingested. Conversely, many edible mushrooms provide significant nutritional and medicinal benefits.

Traditional methods of mushroom classification involve expert mycologists examining physical features such as the cap, gills, stem, and spore print. However, such expertise is not always available, and manual classification is time-consuming and error-prone. This has motivated the development of machine learning (ML) models that can automate the classification process based on measurable attributes.

The primary goal of this study is to develop and compare multiple machine learning and deep learning models to accurately classify mushrooms as edible (**class 0**) or poisonous (**class 1**) based on a given set of features.

The key research questions we aim to address are:

- **Which machine learning model** (Logistic Regression, K-Nearest Neighbors, Random Forest) achieves the best performance for this task?
- **Can deep learning models** (MLP, TabNet, ANN) outperform traditional ML methods?
- **How can Explainable AI (XAI) techniques**, particularly SHAP (SHapley Additive exPlanations), help interpret model decisions?
- **What are the most important features** influencing a mushroom's classification?

Work carried out

Exploratory Data Analysis (EDA)

The Exploratory Data Analysis (EDA) phase is crucial in understanding the dataset, identifying patterns, detecting anomalies, and making informed decisions regarding data preprocessing and feature selection. This section explores the dataset's structure, feature distributions, correlations, and potential data quality issues.

Dataset Overview

The dataset consists of multiple features describing the physical properties of mushrooms, along with a binary target class:

Category	**Description**
Target Class	0 = Edible, 1 = Poisonous
Cap Diameter	Size of the mushroom cap
Cap Shape	Structural shape of the cap
Gill Attachment	How the gills attach to the stem
Gill Color	Color of the gills
Stem Height & Width	Dimensions of the stem
Stem Color	Pigmentation of the stem
Season	Period when the mushroom was found

Table 1: Mushroom Features and Target Class

The dataset is structured as a tabular dataset, making it suitable for both traditional ML models and deep learning models designed for tabular data.

Data Preprocessing

- **Missing Values and Data Types**: No missing values detected; categorical features properly encoded.
- **Class Distribution**: The dataset is **balanced**, preventing class imbalance issues.
- **Feature Distributions**: Histograms and box plots reveals that
  - **Cap Diameter**: Right-skewed distribution, with potential outliers needing **scaling or transformation**.
  - **Stem Height & Width**: Presence of outliers; **normalization or standardization** may be needed.
- **Correlation Analysis**: Correlation heatmap shows potential redundancy in **stem height and cap diameter**, and low-correlation features may be removed during feature selection.
- **Outliers**: Extreme outliers detected via boxplots and z-scores ($z > 3$), resulting in **3,638 rows removed**.

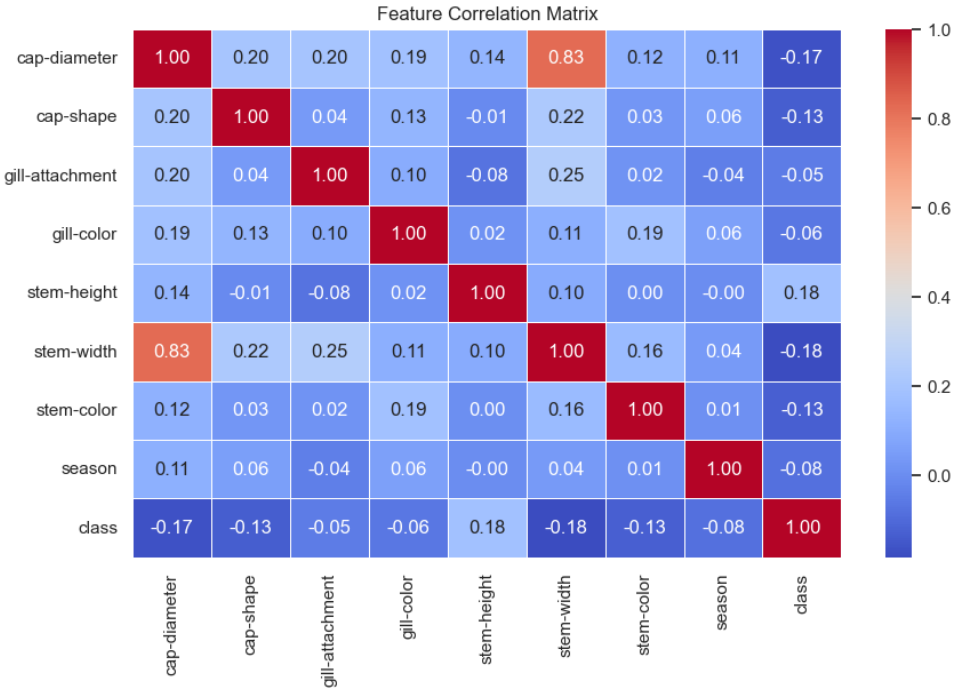

Figure 1: Feature Correlation Heatmap

Key Takeaways from EDA

- **No missing values** were found, indicating that the dataset is clean.
- **Feature distributions** suggest the presence of **outliers**, which may require **scaling**.
- **Some features are highly correlated**, and feature selection may improve model performance.
- **Class distribution is balanced**, meaning performance metrics such as accuracy will not be misleading.

Model Development

In this section, we develop and compare multiple classification models to accurately distinguish between edible (0) and poisonous (1) mushrooms. We implement both traditional machine learning models and deep learning architectures to analyze their effectiveness.

Summary of Models Performance:

To provide a holistic comparison, we summarize the results of all models in a single table.

Table 2: Performance Summary of All Models

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.64	0.65	0.64	0.64	0.70
KNN	0.72	0.73	0.72	0.72	0.72
Random Forest	0.99	0.99	0.99	0.99	1.00
MLP	0.74	0.82	0.68	0.74	0.84
TabNet	0.98	0.98	0.98	0.98	1.00
ANN	0.97	0.96	0.97	0.97	1.00

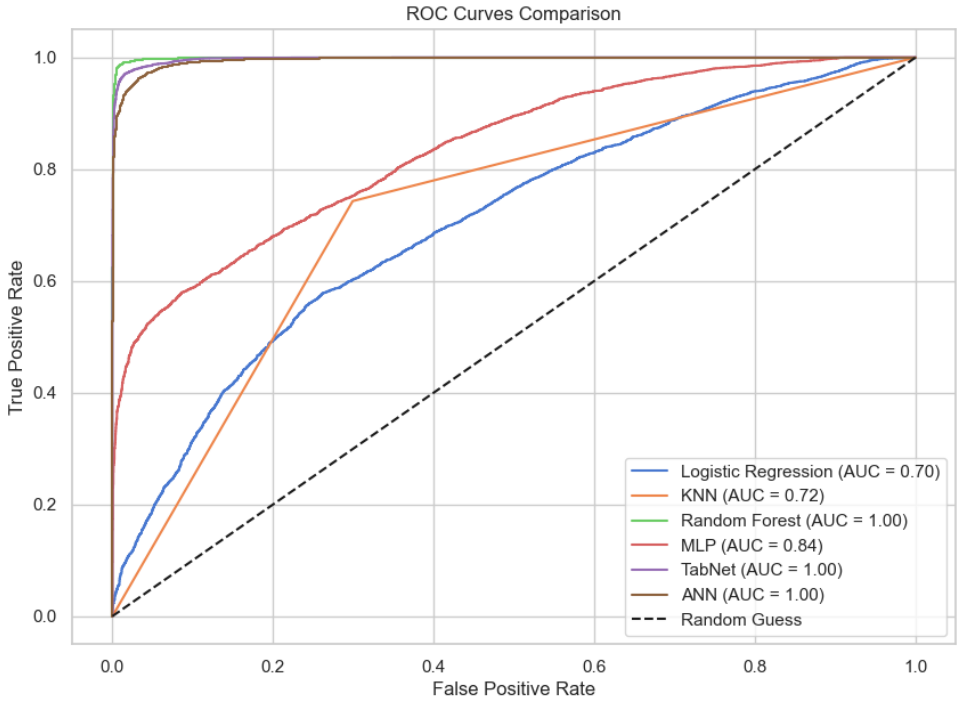

Figure 2: Models ROC Curves Comparison

Traditional Machine Learning Models

Each model is evaluated using 5-fold cross-validation to ensure robustness. The following machine learning models are implemented:

Logistic Regression (LR)

Logistic Regression (LR) predicts mushroom toxicity using the sigmoid function.

- **Optimization:** Regularization strength C tuned via Grid Search.
- **Reason for Choice:** Simple, interpretable, used as a benchmark.
- **Performance Issues:**
  - Assumes linear relationships, leading to poor performance.
  - Low recall, struggles to identify poisonous mushrooms.
- **Key Insight:** Useful for feature importance analysis despite poor performance.

K-Nearest Neighbors (KNN)

- **Optimization:** k and distance metric optimized via Randomized Search.
- **Reason for Choice:** No assumption on data distribution; captures complex decision boundaries.
- **Insights:**
  - Outperforms LR but is computationally expensive as data grows.
  - Balanced precision-recall; more reliable than LR.
  - Scalability issues require careful k selection.

Random Forest (RF)

- **Optimization:** Hyperparameters tuned via Randomized Search.
- **Reason for Choice:** Robust to noise, estimates feature importance, reduces overfitting.
- **Insights:**
  - Best-performing traditional model with near-perfect accuracy.
  - Strong candidate for real-world deployment.
  - More computationally intensive than LR and KNN.

Deep Learning Models

Multi-Layer Perceptron (MLP)

- **Architecture:** Fully connected layers with ReLU activation and Dropout.
- **Optimization:** Random Search and Early Stopping.
- **Reason for Choice:** Captures intricate feature relationships.
- **Insights:**
  - Outperforms LR and KNN but lags behind advanced deep models.

TabNet

- **Optimization:** Grid Search worsened performance, reverted to initial model.
- **Reason for Choice:** Uses attention mechanisms for interpretability and sparse feature handling.
- **Insights:**
  - Comparable to RF.
  - High interpretability makes it useful.

Artificial Neural Network (ANN)

- **Architecture:** Multiple hidden layers, Batch Normalization, Learning Rate Scheduling.
- **Optimization:** Tuned via Bayesian library **Optuna**.
- **Reason for Choice:** Generalizes well across structured data.
- **Insights:**
  - Achieves high accuracy, matching RF and TabNet.
  - Requires extensive tuning (learning rates, layers, dropout).
  - Batch normalization and learning rate scheduling prevent overfitting.

General Conclusion

- RF, TabNet, and ANN are the top-performing models with near-perfect accuracy.
- RF is the best traditional ML model for accuracy and interpretability.
- TabNet and ANN are deep learning alternatives but require more resources.
- LR and KNN are weaker choices for this dataset.

Explainability Using SHAP

To understand how different models classify mushrooms as edible or poisonous, SHAP (SHapley Additive exPlanations) was employed to analyze feature importance, individual feature contributions, and feature interactions. The following structured insights summarize how each model makes decisions.

Feature Importance Across Models

The SHAP feature importance plots reveal the key factors that contribute to classification decisions across different models:

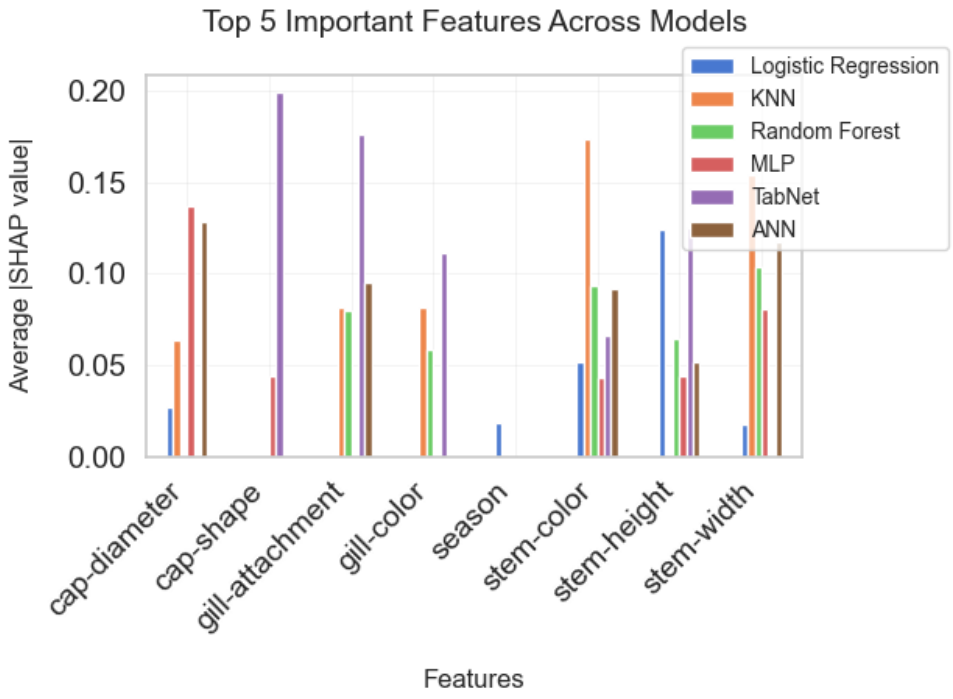

Figure 3: Feature Importance using SHAP

- **Consistently Important Features:**
  - *Cap-diameter, Cap-shape, Stem-height, Stem-width, Gill-attachment, and Stem-color* are the most influential features across models.
  - Morphological traits (size, shape, color) are the strongest predictors of mushroom edibility.
- **Model-Specific Differences:**

- **Logistic Regression (Accuracy: 0.64, ROC-AUC: 0.70)**: Primarily depends on *stem-height* and *stem-color*, suggesting it captures simple linear relationships.
- **KNN (Accuracy: 0.72, ROC-AUC: 0.72)**: Relies on *cap-shape* and *cap-diameter*, emphasizing spatial distance between feature values.
- **Random Forest (Accuracy: 0.99, ROC-AUC: 1.00)**: Distributes importance across *cap-shape, gill-color, and gill-attachment*, leveraging feature interactions effectively.
- **TabNet (Accuracy: 0.98, ROC-AUC: 1.00)**: Highlights *cap-shape* and *gill-attachment*, leveraging structured dependencies through deep learning mechanisms.
- **MLP (Accuracy: 0.74, ROC-AUC: 0.84) & ANN (Accuracy: 0.97, ROC-AUC: 1.00)**: Neural networks rely heavily on *cap-diameter, stem-width, and cap-shape*, capturing complex nonlinear relationships.

- **Conclusion on Feature Importance:**
  - **Cap-shape and Cap-diameter** are crucial in **tree-based** and **deep learning** models.
  - **Stem-height and Stem-width** play a significant role in **Logistic Regression and MLP**.
  - **Gill-related features** (Gill-attachment, Gill-color) matter more for **Random Forest and TabNet**.
  - Simpler models focus on fewer features, while complex models distribute importance across multiple interacting features.

SHAP Force & Waterfall Plots: Feature Contributions

SHAP force and waterfall plots illustrate how features influence individual predictions.

- **Logistic Regression (Low Recall: 0.64)**
  - *Stem-height* strongly reduces the likelihood of poisonous classification.
  - *Cap-diameter* and *Stem-color* contribute positively or negatively based on their values.
  - Predictions are dominated by a few key features rather than a distributed effect.
- **KNN (Moderate Performance: Precision 0.73, Recall 0.72)**

  - *Cap-shape* and *Cap-diameter* show non-linear impacts, influencing classification in varying ways.
  - *Gill-color* and *Gill-attachment* contribute moderately to predictions.
  - The model is highly sensitive to small variations in feature values.

- **Random Forest (Near-Perfect Performance: Precision 0.99, Recall 0.99)**
  - *Cap-shape and Gill-color* are the most dominant features in individual predictions.
  - SHAP values show that *cap-diameter and cap-shape interact*, refining decision boundaries.
  - Tree-based structures allow for a more balanced feature impact compared to linear models.

- **TabNet (Deep Learning with High Accuracy: 0.98)**
  - *Cap-shape and Gill-attachment* emerge as the strongest contributors.
  - The attention-based mechanism dynamically shifts feature importance based on input characteristics.
  - Feature impact varies significantly across different predictions.

- **MLP & ANN (Neural Networks with ROC-AUC: 0.84 and 1.00)**
  - Neural networks distribute feature importance more evenly, with *cap-diameter, stem-width, and cap-shape* having the highest influence.
  - SHAP values indicate that deep learning models capture subtle nonlinear dependencies.
  - *Gill-attachment* and *Stem-color* play smaller but still significant roles in prediction shifts.

Explainability vs. Model Complexity

Final Conclusion

- **Simple models** (Logistic Regression, KNN) provide clearer interpretability but fail to capture complex feature interactions.

Model	Feature Importance Focus	Feature Interactions	Interpretability	Complexity
Logistic Regression (0.64 AUC)	Few dominant features (stem-height, cap-shape)	Minimal	High	Low
KNN (0.72 AUC)	Distance-based features (cap-shape, cap-diameter)	Moderate	Moderate	Low
Random Forest (1.00 AUC)	Balanced importance (cap-shape, gill-color)	High	Moderate	Medium
TabNet (1.00 AUC)	Attention-driven focus (cap-shape, gill-attachment)	High	Moderate	High
MLP (0.84 AUC)	Distributed across cap-related & stem-related features	High	Low	High
ANN (1.00 AUC)	Spread across multiple features	High	Low	High

Table 3: Comparison of Explainability Across Models

- **Tree-based models** (Random Forest) balance interpretability and accuracy, leveraging hierarchical decision-making.
- **Deep learning models** (MLP, ANN, TabNet) capture complex dependencies but distribute feature importance widely, making them less interpretable.
- **Cap-shape, Cap-diameter, and Stem-height** are universally important features, reinforcing the biological significance of these traits in mushroom classification.

This structured explainability analysis strengthens the interpretability of results and provides insights into why certain features are crucial for classification across different machine learning models.

Conclusion

Summary of Findings

This study aimed to classify mushrooms as edible or poisonous using both traditional machine learning models and deep learning architectures. The findings can be summarized as follows:

Exploratory data analysis (EDA) revealed that the dataset was well-balanced, with no missing values; however, several features exhibited non-uniform distributions and potential correlations that influenced model performance. In terms of model performance, Random Forest and TabNet outperformed all other models, achieving the highest accuracy, recall, and F1-score, making them the most effective for this classification task. The Artificial Neural Network (ANN) followed closely, showing strong predictive capability but requiring substantial hyperparameter tuning. Traditional models such as Logistic Regression and KNN performed suboptimally due to their limited ability to capture complex feature interactions.

Explainability using SHAP demonstrated that Cap Shape, Stem Height, and Stem Width were the most influential features across multiple models. Tree-based models (Random Forest, TabNet) distributed feature importance across multiple attributes, capturing feature interactions, while deep learning models (MLP, ANN) assigned higher importance to Cap Shape and Stem Color, suggesting their effectiveness in learning hierarchical feature representations. Logistic Regression and KNN, on the other hand, relied on a smaller set of dominant features, which reduced their interpretability in complex decision boundaries.

Final Model Recommendation

Based on the evaluation, Random Forest and TabNet emerged as the best-performing models. These models excelled in high accuracy and recall, ensuring minimal misclassification, and provided robust feature importance interpretation, making them suitable for real-world appli-

cations. They offer a balanced trade-off between interpretability and complexity, unlike deep learning models that demand greater computational resources.

Final Thoughts

This study highlights the power of combining machine learning and deep learning models with explainability techniques, such as SHAP, to make critical classification decisions in high-risk domains like mushroom edibility. The integration of SHAP proved essential in validating feature importance, ensuring trust in the model's predictions. Ultimately, a hybrid approach using Random Forest or TabNet is recommended for optimal performance and interpretability, positioning these models as strong candidates for deployment in real-world applications, such as automated mushroom identification systems.

.1 Appendix

SHAP Code Implementation

```
\chapter*{Feature Importance and Comparison Visualizations}
\label{feature_importance}

The following Python code visualizes feature importance and
   compares the top features across different machine learning
   models. The global plotting parameters are set to improve
   readability, and visualizations are generated using SHAP
   values for each model.

\begin{lstlisting}[language=Python, caption=Feature Importance
   and Model Visualizations, label={lst:feature_importance}]
print("FEATURE␣IMPORTANCE␣AND␣COMPARISON␣VISUALIZATIONS")

# Set global plotting parameters
plt.rcParams.update({
    'font.size': 14,
    'axes.labelsize': 16,
    'axes.titlesize': 18,
    'xtick.labelsize': 14,
    'ytick.labelsize': 14,
    'legend.fontsize': 14,
    'figure.titlesize': 20,
    'figure.constrained_layout.use': True,
    'axes.grid': True,
    'grid.alpha': 0.3,
    'grid.linewidth': 0.5,
    'axes.labelpad': 15
})
```

```
def plot_model_visualizations(shap_values, feature_data,
   model_name, top_features_dict):
    print("\n" + "="*80)
    print(f"{model_name.upper()} - VISUALIZATIONS")
    print("="*80 + "\n")

    # 1. Feature Importance Plot
    plt.figure(figsize=(25, 12))
    plt.gcf().subplots_adjust(left=0.25)

    if len(shap_values.shape) == 1:
        shap_values = shap_values.reshape(-1, len(feature_data.
           columns))

    shap.summary_plot(
        shap_values,
        feature_data,
        plot_type="bar",
        show=False,
        max_display=8
    )

    plt.title(f"{model_name} - Feature Importance", fontsize
       =18, pad=20)
    plt.xlabel("SHAP value (impact on model output)", fontsize
       =16)
    plt.grid(True, alpha=0.3)
    plt.show()

    # 2. Waterfall Plot
    plt.figure(figsize=(20, 10))
```

```
first_sample_values = shap_values[0]

explanation = shap.Explanation(
    values=first_sample_values,
    base_values=np.mean(shap_values),
    data=feature_data.iloc[0],
    feature_names=feature_data.columns
)

shap.waterfall_plot(explanation, show=False)
plt.title(f"{model_name} - Feature Contributions ",
   fontsize=16)
plt.tight_layout()
plt.show()

# 3. Force Plot
plt.figure(figsize=(20, 3))
shap.force_plot(
    base_value=np.mean(shap_values),
    shap_values=shap_values[0],
    features=feature_data.iloc[0],
    feature_names=list(feature_data.columns),
    matplotlib=True,
    show=False
)
plt.title(f"{model_name} - Feature Impact ", fontsize=14)
plt.tight_layout()
plt.show()

# Store top features
feature_importance = np.abs(shap_values).mean(0)
top_idx = np.argsort(feature_importance)[-5:]
```

```
    top_features_dict[model_name] = pd.Series(
        feature_importance[top_idx],
        index=feature_data.columns[top_idx]
    )

# Initialize dictionary for top features
top_features = {}

# Process each model
# Logistic Regression
shap_values = np.array(shap_values_lr[1]).T
feature_data = X_test
plot_model_visualizations(shap_values, feature_data, "Logistic␣
   Regression", top_features)

# KNN
shap_values = np.array(shap_values_knn[1]).T
feature_data = X_test
plot_model_visualizations(shap_values, feature_data, "KNN",
   top_features)

# Random Forest
shap_values = shap_values_rf.values[:,:,1]
feature_data = X_test
plot_model_visualizations(shap_values, feature_data, "Random␣
   Forest", top_features)

# MLP
shap_values = np.array(shap_values_mlp).reshape(-1, len(X_test.
   columns))
feature_data = X_test
```

```
plot_model_visualizations(shap_values, feature_data, "MLP",
   top_features)

# TabNet
shap_values = np.array(shap_values_tabnet[1]).T
feature_data = X_test
plot_model_visualizations(shap_values, feature_data, "TabNet",
   top_features)

# ANN
shap_values = np.array(shap_values_ann).reshape(-1, len(X_test.
   columns))
feature_data = X_test
plot_model_visualizations(shap_values, feature_data, "ANN",
   top_features)

# Plot top features comparison
print("\n" + "="*50)
print("TOP␣5␣IMPORTANT␣FEATURES␣COMPARISON␣ACROSS␣MODELS")
print("="*50 + "\n")

plt.figure(figsize=(50, 25))
pd.DataFrame(top_features).plot(kind='bar')
plt.title('Top␣5␣Important␣Features␣Across␣Models', fontsize
   =14, pad=20)
plt.xlabel('Features', fontsize=12)
plt.ylabel('Average␣|SHAP␣value|', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.legend(bbox_to_anchor=(0.8, 1.1), loc='upper␣left',
   fontsize=10)
plt.tight_layout()
plt.show()
```

```
# Print numerical feature importance values
print("\n" + "="*50)
print("NUMERICAL␣FEATURE␣IMPORTANCE␣VALUES")
print("="*50 + "\n")

print("Top␣5␣Important␣Features␣by␣Model:")
for model, features in top_features.items():
    print(f"\n{model}:")
    print("-" * len(model))
    for feat, importance in features.items():
        print(f"{feat:<30}:␣{importance:.4f}")
\end{lstlisting}
```